PROJECT SHEET 3 · FROM "STUDENT PROJECT" TO "HIRE SIGNAL"

Upgrade Roadmap for Top-Tier DV Interviews

What to add to the repo, in priority order, and what big silicon DV interview loops commonly test.

P0 correctness — do first
P1 strongest interview signal
P2 industry methodology
P3 stand-out extras

1 Roadmap

Pri	Add	What it demonstrates	Interview question it prepares you for
P0	Make failures fatal; check pass count > 0	You trust nothing that can't fail	"How does your regression decide pass/fail?"
P0	Fix W-before-AW deadlock + SLVERR (bugs #2, #3)	Protocol knowledge, real bug-finding	"Tell me about a bug you found."
P0	Get the UVM layer compiling and running on a real simulator (EDA Playground has free UVM)	Your resume claim is true	"Walk me through your env's build/connect phases."
P1	SVA protocol checker, bound to the DUT	Assertions, bind, temporal logic	"Write an assertion for the VALID/READY rule."
P1	Driver knobs: random AW/W skew, BREADY/RREADY back-pressure (config object)	Stimulus that hunts corner cases	"How would you find a deadlock?"
P1	Protocol coverage: handshake order, stall lengths, resp × direction, all 15 strobes	Coverage plan driven by risk, not by fields	"You're at 100% coverage but still found a bug. Why?"
P2	UVM RAL model + AXI-Lite adapter + predictor	AXI-Lite is a register bus — RAL is how industry tests it	"Frontdoor vs backdoor? Explicit vs implicit prediction?"
P2	Regression script: N seeds, log parsing, merged coverage, failure triage	Scale and automation (Python/Make)	"Test fails 1 in 1000 seeds. What do you do?"
P3	Formal: prove the SVA properties (e.g. SymbiYosys free, or JasperGold/VC Formal at work)	Formal mindset — growing fast in CPU/GPU teams	"Which blocks suit formal, and why?"
P3	Full AXI4 slave: bursts, IDs, outstanding (sheets 5–11 of this canvas)	Real protocol complexity	"Scoreboard with out-of-order responses?"

2 P1 · AXI4-Lite SVA checker (bind it, don't edit the RTL)

```
module axi_lite_sva (input logic aclk, aresetn,
  input logic awvalid, awready, wvalid, wready, bvalid, bready,
  input logic arvalid, arready, rvalid, rready,
  input logic [31:0] awaddr, wdata, rdata, input logic [3:0] wstrb,
  input logic [1:0]  bresp, rresp);

  default clocking cb @(posedge aclk); endclocking
  default disable iff (!aresetn);

  // VALID held + payload stable until READY (all 5 channels)
  a_aw: assert property (awvalid && !awready |=> awvalid && $stable(awaddr));
  a_w : assert property (wvalid && !wready |=> wvalid && $stable({wdata, wstrb}));
  a_b : assert property (bvalid && !bready |=> bvalid && $stable(bresp));
  a_r : assert property (rvalid && !rready |=> rvalid && $stable({rdata, rresp}));
  a_x : assert property (!$isunknown({awvalid, wvalid, bvalid, arvalid, rvalid}));

  // B only after both AW and W were accepted; then within 8 cycles
  logic [7:0] aw_n, w_n, b_n;
  always_ff @(posedge aclk or negedge aresetn)
    if (!aresetn) {aw_n, w_n, b_n} <= '0;
    else begin
      aw_n <= aw_n + (awvalid && awready);
      w_n  <= w_n  + (wvalid  && wready);
      b_n  <= b_n  + (bvalid  && bready);
    end
  a_b_order: assert property (bvalid |-> aw_n != b_n && w_n != b_n);
  a_b_live : assert property (aw_n != b_n && w_n != b_n |-> ##[0:8] bvalid);
endmodule

bind axi4_slave_ram axi_lite_sva u_sva (.*);   // port names match
```

`a_b_live` fires on the original RTL in the W-before-AW case — the assertion would have caught bug #2 on day one.

3 P1 · Driver knobs

```
class axi_cfg extends uvm_object;
  `uvm_object_utils(axi_cfg)
  rand int unsigned max_skew  = 4;  // AW vs W start delay
  rand int unsigned max_bstall = 6;  // BREADY low cycles
  rand int unsigned max_rstall = 6;  // RREADY low cycles
  function new(string name = "axi_cfg"); super.new(name); endfunction
endclass

// in drive_write(): channels start independently
fork
  begin repeat ($urandom_range(0, cfg.max_skew)) @(vif.driver_cb);
        drive_aw(txn); end
  begin repeat ($urandom_range(0, cfg.max_skew)) @(vif.driver_cb);
        drive_w(txn);  end
join
repeat ($urandom_range(0, cfg.max_bstall)) @(vif.driver_cb);
vif.driver_cb.bready <= 1;
```

Set via `uvm_config_db#(axi_cfg)::set` from the test, so one test can be "no stalls" and another "heavy back-pressure" without touching the driver.

4 P1 · Protocol coverage (sample in the monitor)

```
typedef enum {AW_FIRST, W_FIRST, SAME_CYCLE} order_e;
covergroup cg_proto with function sample(order_e ord,
                        int bstall, int rstall, bit wr, bit [1:0] resp);
  cp_ord : coverpoint ord;
  cp_bst : coverpoint bstall { bins none = {0}; bins short_ = {[1:3]};
                               bins long_ = {[4:$]}; }
  cp_rst : coverpoint rstall { bins none = {0}; bins short_ = {[1:3]};
                               bins long_ = {[4:$]}; }
  cp_resp: coverpoint resp   { bins okay = {0}; bins slverr = {2}; }
  x_ord_bst : cross cp_ord, cp_bst;
  x_dir_resp: cross wr, cp_resp;
endgroup
```

5 P2 · RAL — the standard way to verify an AXI-Lite register/memory block

```
class axi_lite_adapter extends uvm_reg_adapter;
  `uvm_object_utils(axi_lite_adapter)
  function new(string name = "axi_lite_adapter"); super.new(name); endfunction

  virtual function uvm_sequence_item reg2bus(const ref uvm_reg_bus_op rw);
    axi4_seq_item t = axi4_seq_item::type_id::create("t");
    t.txn_type = (rw.kind == UVM_WRITE) ? axi4_seq_item::AXI4_WRITE
                                        : axi4_seq_item::AXI4_READ;
    t.addr = rw.addr;  t.data = rw.data;  t.wstrb = 4'hF;
    return t;
  endfunction

  virtual function void bus2reg(uvm_sequence_item item, ref uvm_reg_bus_op rw);
    axi4_seq_item t;
    if (!$cast(t, item)) `uvm_fatal("ADPT", "bad item type")
    rw.kind   = (t.txn_type == axi4_seq_item::AXI4_WRITE) ? UVM_WRITE : UVM_READ;
    rw.addr   = t.addr;
    rw.data   = (rw.kind == UVM_READ) ? t.rdata : t.data;
    rw.status = (((rw.kind == UVM_WRITE) ? t.bresp : t.rresp) == 2'b00)
                ? UVM_IS_OK : UVM_NOT_OK;
  endfunction
endclass
```

- **Model:** `uvm_reg_block` with a `uvm_mem` (256 × 32) — or a few `uvm_reg`s if you add CSRs.
- **Connect:** `map.set_sequencer(agent.sequencer_h, adapter)`.
- **Predictor:** `uvm_reg_predictor` fed by the monitor → **explicit prediction**, mirror tracks bus traffic from any master.
- **Built-in sequences:** `uvm_reg_hw_reset_seq`, `uvm_reg_bit_bash_seq`, `uvm_mem_walk_seq` — free tests.
- **Backdoor:** `add_hdl_path` → peek/poke the RAM without bus traffic, useful to preload or to check.

Know cold: `write/read` (bus) vs `set/get` (desired value) vs `update/mirror` vs `peek/poke` (backdoor).

6 What big silicon DV interview loops commonly cover

Based on widely shared candidate experiences; exact rounds vary by team and level — ask your recruiter for the format.

SystemVerilog & UVM depth

OOP (virtual methods, polymorphism, deep vs shallow copy), constraints & `solve…before`, coverage, factory, config_db, phases, TLM, sequences.

Assertions & formal

Write SVA live: handshakes, one-hot, req→ack within N, FIFO never overflows. When formal beats simulation.

"Verify this block"

Async FIFO, arbiter, cache, AXI slave, credit-based link. Testplan → checker → coverage → corner cases.

Architecture & protocols

AXI/AHB/APB, caches & coherency (MESI/MOESI), memory ordering, pipelines and hazards, CDC/RDC. Especially deep at CPU/GPU companies.

Debug scenarios

Fails in batch not GUI; 1-in-1000 seed; X propagation; race conditions; "scoreboard mismatch — DUT or model?"

Coding

C/C++ or Python data-structure problems (some loops include a general software round), plus SV coding: constraints, a small scoreboard, a reference model.

Project deep-dive is near-universal: expect 20–40 minutes on one resume project, with "why" after every answer. This canvas's bugs sheet is your preparation for that round.

2-week plan: days 1–3 P0 fixes + push; 4–6 SVA + knobs + coverage; 7–9 RAL; 10–11 regression script; 12–14 mock interviews on sheets 1–4 of this row.